UNIVERSIDAD DON BOSCO	**UNIVERSIDAD DON BOSCO FACULTAD DE INGENIERÍA ESCUELA DE COMPUTACIÓN**	**CICLO 02-2026**
	MATERIA | **DESARROLLO DE SOTWARE PARA MOVILES**	**GUIA DE LABORATORIO # 12**
	PRÁCTICA | **JETPACK COMPOSE**	

I. OBJETIVOS

- Que el estudiante comprenda el desarrollo de una aplicación utilización Compose.
- Que el estudiante aprenda a hacer interfaces declarativas.

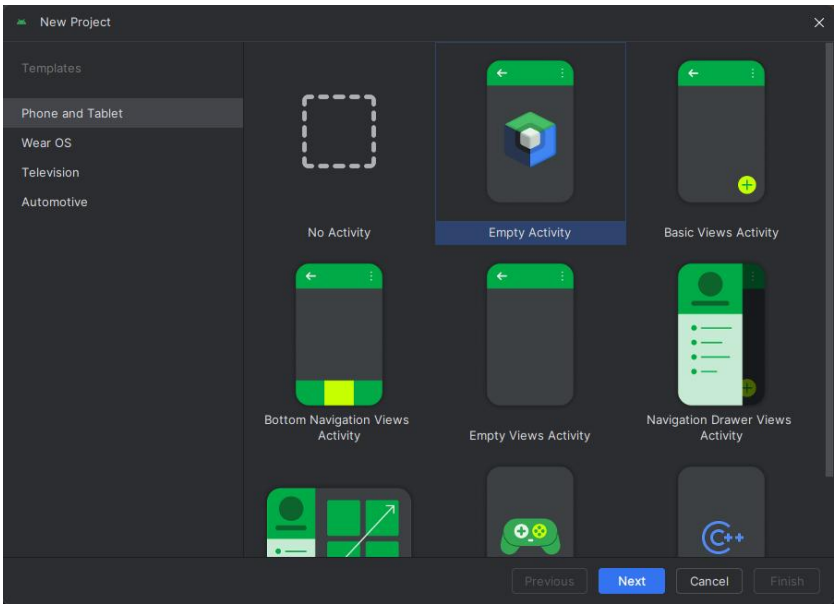
II. INTRODUCCIÓN TEORICA

Jetpack Compose es el kit de herramientas moderno de Android para compilar IU nativas con menos código, herramientas potentes y APIs intuitivas de Kotlin. Simplifica y acelera el desarrollo de IU, incluida la adaptación a cualquier factor de forma, desde smartphones, dispositivos plegables y tablets hasta TVs y wearables.

Compose ofrece componentes de IU precompilados que funcionan con gráficos, animaciones y otros elementos visuales.

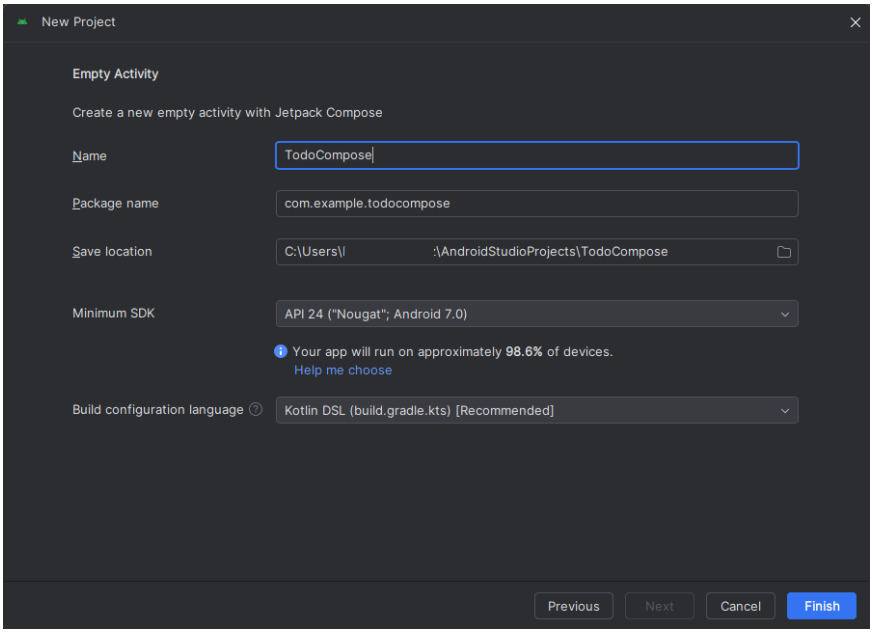
III. DESARROLLO DE LA PRÁCTICA

1. Cree un nuevo proyecto en Android Studio, asegúrese de seleccionar la plantilla **Empty Activity**.



2. Configure el proyecto con las siguientes características:

- **Nombre:** TodoCompose.
- **Ubicación:** por defecto (o use una ubicación de su preferencia).
- **Lenguaje:** Kotlin.
- **Minimun SDK:** API 24 Nougat.



3. Edite el archivo **AndroidManifest.xml** para agregar el permiso de conectividad a internet**.**

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <uses-permission android:name="android.permission.INTERNET"/>

    <application>
        .....
    </application>

</manifest>
```

4. Modifique el archivo **libs.versions.toml** para agregar las librerías necesarias para construir la app. Para ello, cambie la vista de su proyecto de **Android** a **Project**, posteriormente, localiza el archivo **libs.versions.toml** en la carpeta **gradle**.

- **Retrofit 2**: Librería encargada del consumo de las API.
- **Retrofit 2 Converter Gson**: complemento que simplificará el proceso de pasar un JSON a una clase Kotlin.

En la sección **[versions]**, defina las siguientes versiones de las herramientas.

- **retrofit = "2.9.0"**

En la sección **[libraries]** defina las librerías con sus respectivos nombres y referencias a las versiones previamente escritas.

- **retrofit = { module = "com.squareup.retrofit2:retrofit", version.ref = "retrofit" }**
- **retrofit-converter-gson = { module = "com.squareup.retrofit2:converter-gson", version.ref = "retrofit" }**

```toml
[versions]
agp = "8.11.2"
kotlin = "2.0.21"
coreKtx = "1.17.0"
junit = "4.13.2"
junitVersion = "1.3.0"
espressoCore = "3.7.0"
lifecycleRuntimeKtx = "2.9.4"
activityCompose = "1.11.0"
composeBom = "2024.09.00"
retrofit = "2.10.0"

[libraries]
androidx-core-ktx = { group = "androidx.core", name = "core-ktx", version.ref = "coreKtx" }
junit = { group = "junit", name = "junit", version.ref = "junit" }
androidx-junit = { group = "androidx.test.ext", name = "junit", version.ref = "junitVersion" }
androidx-espresso-core = { group = "androidx.test.espresso", name = "espresso-core", version.ref = "espressoCore" }
androidx-lifecycle-runtime-ktx = { group = "androidx.lifecycle", name = "lifecycle-runtime-ktx", version.ref = "lifecycleRuntimeKtx" }
androidx-activity-compose = { group = "androidx.activity", name = "activity-compose", version.ref = "activityCompose" }
androidx-compose-bom = { group = "androidx.compose", name = "compose-bom", version.ref = "composeBom" }
androidx-ui = { group = "androidx.compose.ui", name = "ui" }
androidx-ui-graphics = { group = "androidx.compose.ui", name = "ui-graphics" }
androidx-ui-tooling = { group = "androidx.compose.ui", name = "ui-tooling" }
androidx-ui-tooling-preview = { group = "androidx.compose.ui", name = "ui-tooling-preview" }
androidx-ui-test-manifest = { group = "androidx.compose.ui", name = "ui-test-manifest" }
androidx-ui-test-junit4 = { group = "androidx.compose.ui", name = "ui-test-junit4" }
androidx-material3 = { group = "androidx.compose.material3", name = "material3" }
retrofit = { module = "com.squareup.retrofit2:retrofit", version.ref = "retrofit" }
converter-gson = { module = "com.squareup.retrofit2:converter-gson", version.ref = "retrofit" }
```

5. Modifique el archivo **build.gradle.kts (Module :app)** para agregar los nombres definidos previamente en el archivo **libs.versions.toml** para implementar las dependencias.

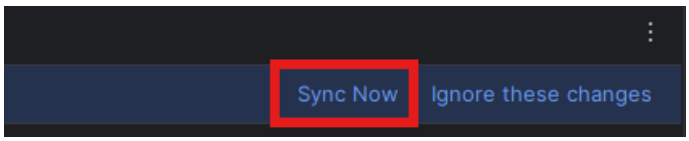
```kotlin
dependencies {

    implementation(libs.androidx.core.ktx)
    implementation(libs.androidx.lifecycle.runtime.ktx)
    implementation(libs.androidx.activity.compose)
    implementation(platform(libs.androidx.compose.bom))
    implementation(libs.androidx.ui)
    implementation(libs.androidx.ui.graphics)
    implementation(libs.androidx.ui.tooling.preview)
    implementation(libs.androidx.material3)
    testImplementation(libs.junit)
    androidTestImplementation(libs.androidx.junit)
    androidTestImplementation(libs.androidx.espresso.core)
    androidTestImplementation(platform(libs.androidx.compose.bom))
    androidTestImplementation(libs.androidx.ui.test.junit4)
    debugImplementation(libs.androidx.ui.tooling)
    debugImplementation(libs.androidx.ui.test.manifest)

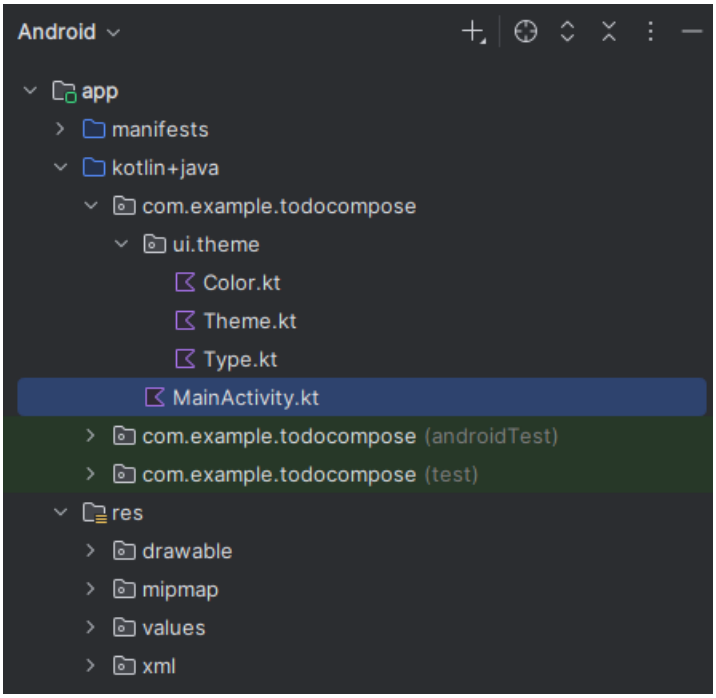
    // Retrofit para hacer peticiones HTTP
    implementation(libs.retrofit)

    // Convertidor Gson para convertir el JSON a objetos Kotlin
    implementation(libs.converter.gson)
}
```

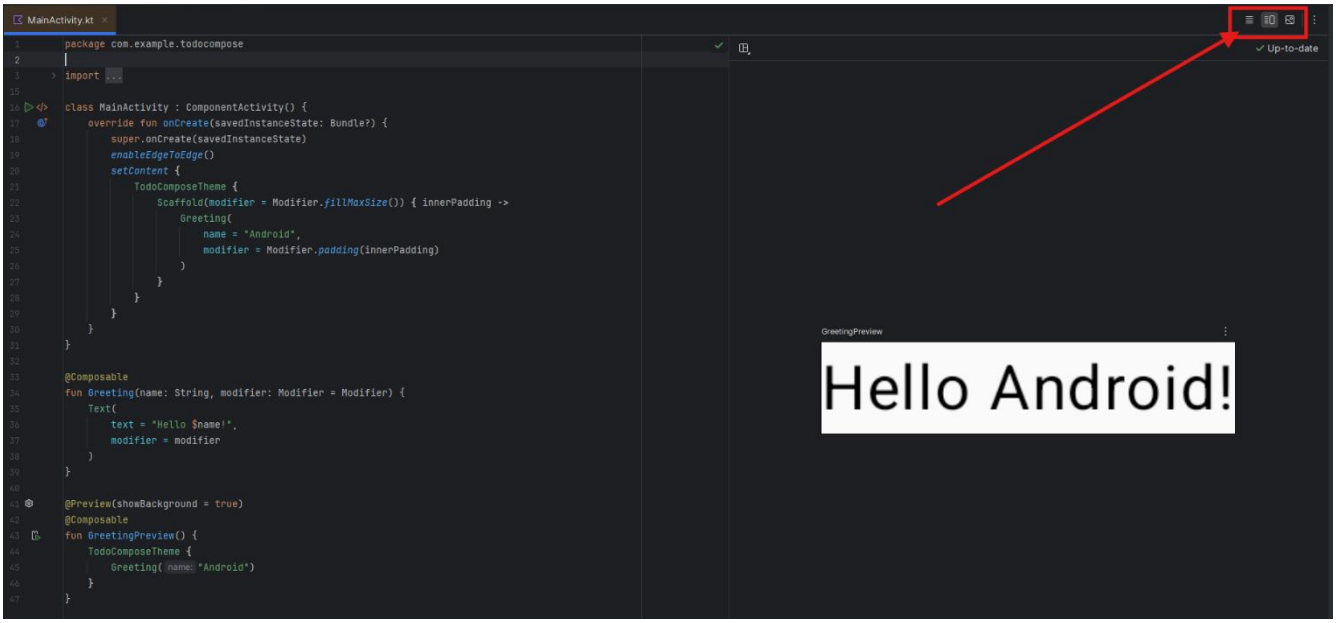
6. Sincronice el proyecto con Gradle dando clic en la opción **Sync Now** o en caso de que no aparezca, cierre y abra nuevamente el proyecto.



7. Una vez finalizada la sincronización del proyecto, observe la estructura de directorios del proyecto. Podrá visualizar que existen clases de Kotlin y archivos de recursos, pero no archivos para vista XML.

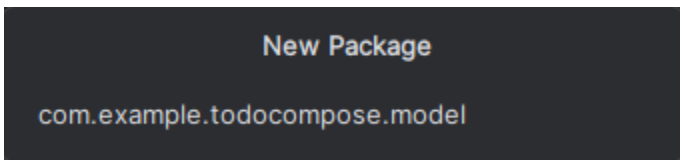


8. Abra el archivo **MainActivity.kt** y seleccione la opción diseño para poder visualizar el código de ejemplo generado por la plantilla.

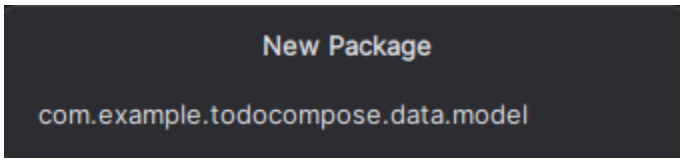


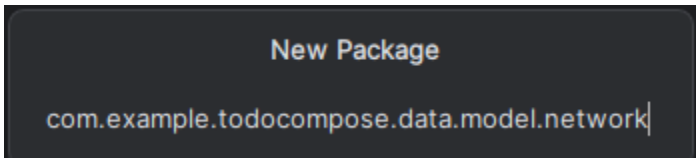
Es posible visualizar el resultado de cada función decorada con el atributo **@Composable** agregando el atributo **@Preview** si la funciona no recibe ningún parámetro, de lo contrario utilice el atributo **@PreviewParameter()**.

9. Dentro del paquete principal **com.example.todocompose** agregue un nuevo paquete con el nombre **data**.

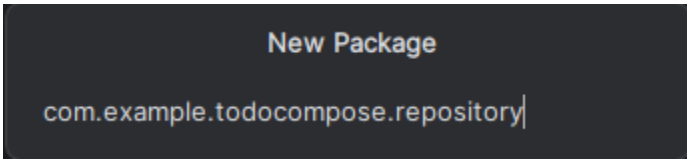


10. Dentro del paquete **data** recién creado, agregue dos nuevos paquetes con el nombre **model** y **network**.





11. Dentro del paquete principal **com.example.todocompose** agregue un nuevo paquete con el nombre **repository**.



12. Una vez configurada la estructura del proyecto para implementar el **patrón MVVM**, procederemos a agregar las clases para su funcionamiento, para ello, sobre el paquete **data.model** agregar una nueva clase de Kotlin con el nombre **Todo** con el siguiente código.

```
package com.example.todocompose.data.model

data class Todo (
    val id: String,
    val title: String,
    val description: String,
    val done: Boolean
)
```

13. En el paquete **data.model.network** agregue la interfaz correspondiente a las llamadas de Retrofit.

```
package com.example.todocompose.data.model.network

import com.example.todocompose.data.model.Todo
import retrofit2.http.*

interface TodoApiService {

    @GET("/to-do")
    suspend fun getTodos(): List<Todo>

}
```

Observe que solo esta declarada la firma del método para obtener la lista de tareas desde la API.

14. Agregue una nueva clase con el nombre **TodoRepository.kt** en el paquete **repository** y modifique el código de acuerdo con la siguiente ilustración.

```
package com.example.todocompose.repository

import com.example.todocompose.data.model.Todo
import com.example.todocompose.data.model.network.TodoApiService

class TodoRepository(private val api: TodoApiService) {
    suspend fun fetchTodos(): List<Todo> {
        return api.getTodos()
    }
}
```

La clase **TodoRepository.kt** será la encargada de obtener el listado de tareas desde la API, por lo que la lógica de negocio estaría separada del acceso a datos.

15. Sobre el paquete principal agregue una nueva clase con el nombre **TodoViewModel.kt** y agregue el siguiente código.

```
package com.example.todocompose

import android.util.Log
import androidx.lifecycle.ViewModel
import androidx.lifecycle.viewModelScope
import com.example.todocompose.data.model.Todo
import com.example.todocompose.repository.TodoRepository
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.launch
import kotlin.uuid.ExperimentalUuidApi
import kotlin.uuid.Uuid

class TodoViewModel(private val repository: TodoRepository) : ViewModel() {

    private val _todos = MutableStateFlow<List<Todo>>(emptyList())
    val todos: StateFlow<List<Todo>> = _todos

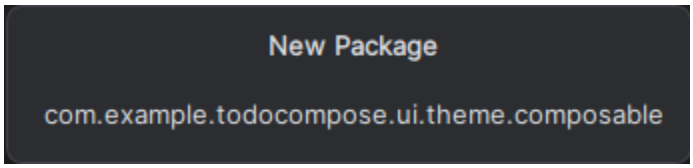
    private val _loading = MutableStateFlow(false)
    val loading: StateFlow<Boolean> = _loading
```

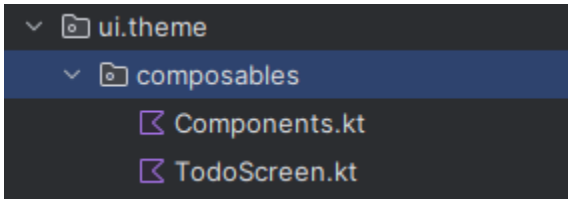
```
    fun loadTodos() {
        viewModelScope.launch {
            _loading.value = true
            try {
                val result = repository.fetchTodos()
                _todos.value = result
            } catch (e: Exception) {
                Log.e("TodoViewModel", e.message.toString())
                _todos.value = emptyList()
            } finally {
                _loading.value = false
            }
        }
    }

}
```

La clase **ViewModel** sirve de puente entre la interfaz de usuario y la capa de acceso a datos de una forma desacoplada y para este ejemplo reactiva. Además, se encarga de exponer el estado a la interfaz de usuario por medio de la variable **todos, esta variable es actualizada automáticamente cuando cambia la lista _todos.**

16. Sobre el paquete **ui.theme** agregue un nuevo paquete con el nombre **composables**, luego, agregue dos clases nuevas con los nombres **TodoScreen.kt** y **Components.kt**.





17. Modifique el código de la clase **Components.kt** con el siguiente código.

```
package com.example.todocompose.ui.theme.composables

import androidx.compose.foundation.layout.*
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
import androidx.compose.material3.*
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
import androidx.compose.ui.tooling.preview.Preview
import androidx.compose.ui.unit.dp
import com.example.todocompose.data.model.Todo
```

```
@Composable
fun TodoList(todos: List<Todo>) {
    LazyColumn(
        modifier = Modifier.fillMaxSize(),
        contentPadding = PaddingValues(vertical = 8.dp),
        verticalArrangement = Arrangement.spacedBy(8.dp)
    ) {
        items(todos) { todo ->
            TodoItem(todo = todo)
        }
    }
}

@Composable
fun TodoItem(todo: Todo) {
    Card(
        elevation = CardDefaults.cardElevation(4.dp),
        modifier = Modifier
            .fillMaxWidth()
            .padding(vertical = 6.dp)
    ) {
        Column(modifier = Modifier.padding(16.dp)) {
            Row(
                horizontalArrangement = Arrangement.SpaceBetween,
                modifier = Modifier.fillMaxWidth()
            ) {
                Text(
                    text = "ID: ${todo.id}",
                    style = MaterialTheme.typography.bodySmall
                )
                Text(
                    text = if (todo.done) "Completada" else "",
                    style = MaterialTheme.typography.bodyMedium,
                    color = if (todo.done) MaterialTheme.colorScheme.primary else
MaterialTheme.colorScheme.error
                )
            }

            Spacer(modifier = Modifier.height(4.dp))

            Text(
                text = "Titulo: ${todo.title}",
                style = MaterialTheme.typography.bodyMedium
            )

            Spacer(modifier = Modifier.height(2.dp))

            Text(
                text = "Descripcion: ${todo.description}",
                style = MaterialTheme.typography.bodySmall
            )
```

```
        }
    }
}
```

En esta clase se definen dos métodos decorados con **@Composable** donde la función **TodoItem** representa una tarjeta individual para mostrar la información de la tarea. El método **TodoList** se encarga de dibujar en la pantalla todo el listado de tareas que recibe por parámetros.

Cada función composable corresponde a un componente individual por lo que se pueden crear (componer) nuevos utilizando los ya existentes.

18. Modifique el código de la clase **TodoScreen.kt** de la siguiente manera.

```
package com.example.todocompose.ui.theme.composables

import androidx.compose.foundation.layout.*
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.tooling.preview.PreviewParameter
import androidx.compose.ui.unit.dp
import com.example.todocompose.TodoViewModel

@Composable
fun TodoScreen(viewModel: TodoViewModel) {
    val todos by viewModel.todos.collectAsState()
    val loading by viewModel.loading.collectAsState()

    var titleInput by remember { mutableStateOf("") }
    var descriptionInput by remember { mutableStateOf("") }

    LaunchedEffect(Unit) {
        viewModel.loadTodos()
    }

    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(16.dp),
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Text(
            text = "Mis Tareas",
            style = MaterialTheme.typography.headlineMedium,
            modifier = Modifier.padding(bottom = 16.dp, top = 16.dp)
        )

        Spacer(modifier = Modifier.height(16.dp))
```

```
        if (loading) {
            CircularProgressIndicator()
        } else {
            TodoList(todos = todos)
        }
    }
}
```

La función **TodoScreen** es el componente principal de la aplicación, este se encarga de mostrar la lista de todas las tareas a través del **ViewModel**.

19. Por último, modifique el código de la clase **MainActivity.kt** como se muestra a continuación, utilice el siguiente enlace para el cliente retrofit https://68163b7232debfe95dbdd500.mockapi.io/academic/v1/.

```
package com.example.todocompose

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import com.example.todocompose.data.model.network.TodoApiService
import com.example.todocompose.repository.TodoRepository
import com.example.todocompose.ui.theme.composables.TodoScreen
import retrofit2.Retrofit
import retrofit2.converter.gson.GsonConverterFactory

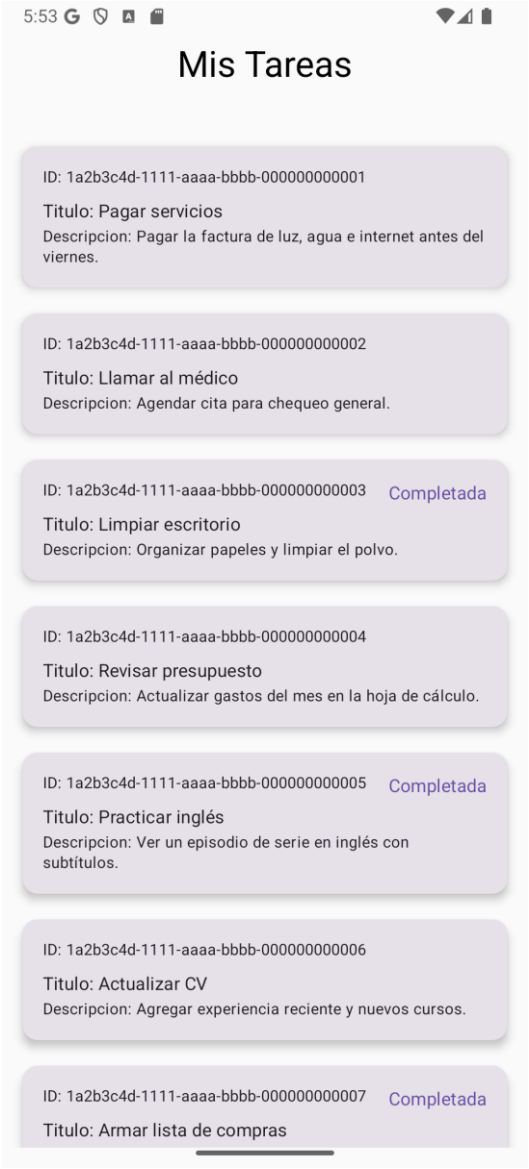
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()

        val retrofit = Retrofit.Builder()
            .baseUrl("https://68163b7232debfe95dbdd500.mockapi.io/academic/v1/")
            .addConverterFactory(GsonConverterFactory.create())
            .build()

        val apiService = retrofit.create(TodoApiService::class.java)
        val repository = TodoRepository(apiService)
        val viewModel = TodoViewModel(repository)

        setContent {
            TodoScreen(viewModel)
        }
    }
}
```

20. Ejecute la aplicación y compruebe los resultados.



Previsualización de una función Composable

Es posible previsualizar el diseño de las funciones composable decorando la función con el atributo **@Preview**.

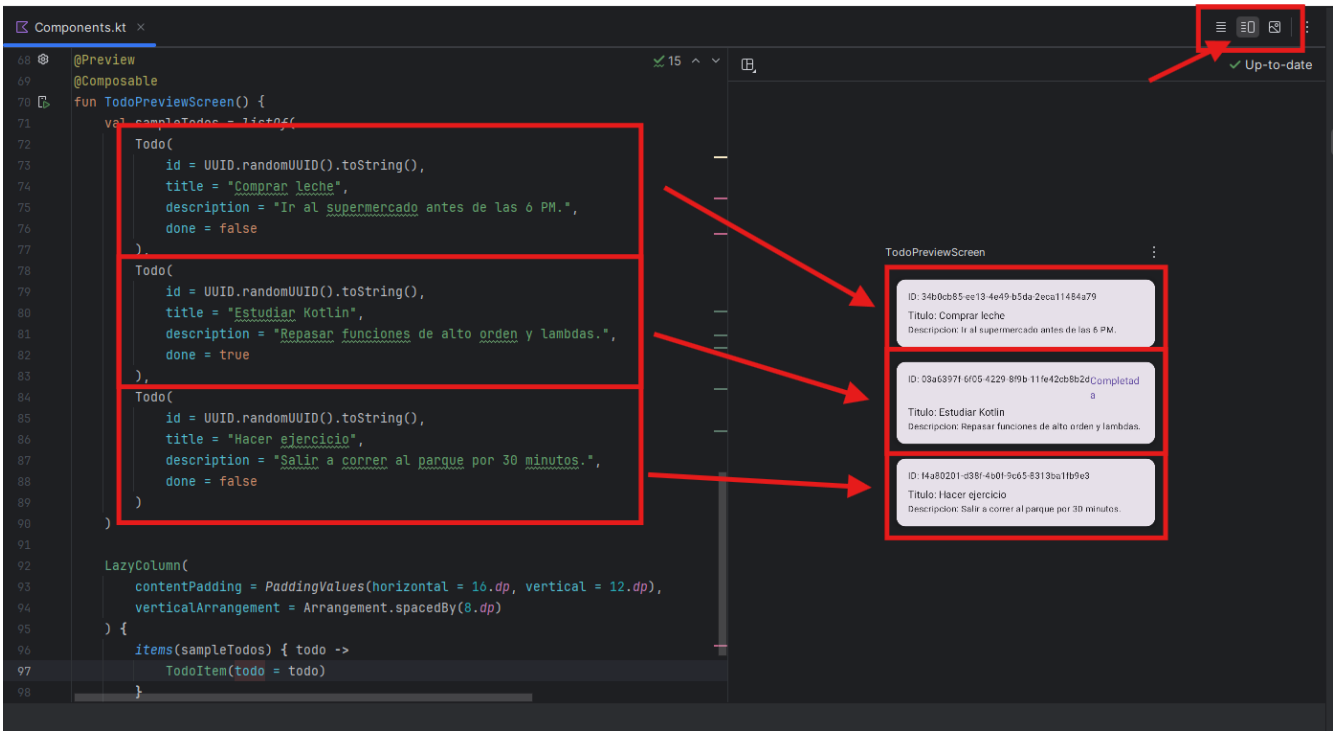
1. Agregue el siguiente método a la clase **Components.kt**.

```
@Preview
@Composable
fun TodoPreviewScreen() {
    val sampleTodos = listOf(
        Todo(
            id = UUID.randomUUID().toString(),
            title = "Comprar leche",
            description = "Ir al supermercado antes de las 6 PM.",
            done = false
        ),
```

```
        Todo(
            id = UUID.randomUUID().ToString(),
            title = "Estudiar Kotlin",
            description = "Repasar funciones de alto orden y lambdas.",
            done = true
        ),
        Todo(
            id = UUID.randomUUID().toString(),
            title = "Hacer ejercicio",
            description = "Salir a correr al parque por 30 minutos.",
            done = false
        )
    )

    LazyColumn(
        contentPadding = PaddingValues(horizontal = 16.dp, vertical = 12.dp),
        verticalArrangement = Arrangement.spacedBy(8.dp)
    ) {
        items(sampleTodos) { todo ->
            TodoItem(todo = todo)
        }
    }
}
```

2. Guarde los cambios y selecciona la opción Split en el panel de visualización para explorar el código y la previsualización del componente al mismo tiempo.



Crear una nueva tarea

1. Modifique la interfaz **TodoApiService** como se muestra a continuación para agregar la firma del método encargado de agregar una nueva tarea.

```
package com.example.todocompose.data.model.network

import com.example.todocompose.data.model.Todo
import retrofit2.http.*

interface TodoApiService {

    @GET("/to-do")
    suspend fun getTodos(): List<Todo>

    @POST("/to-do")
    suspend fun addTodo(@Body todo: Todo): Todo
}
```

2. Modifique la clase **TodoRepository.kt** agregando la implementación del método **addTodo()**.

```
suspend fun addTodo(todo: Todo): Todo {
    return api.addTodo(todo)
}
```

3. Agregue a la clase **TodoViewModel.kt** el siguiente método **addTodo()**.

```
fun addTodo(title: String, description: String) {

    if (title.isBlank()) return

    val newTodo = Todo(
        id = UUID.randomUUID(),
        title = title,
        description = description,
        done = false
    )

    viewModelScope.launch {
        _loading.value = true
        try {
            val result = repository.addTodo(newTodo)
            _todos.value += result
        } catch (e: Exception) {
            Log.e("TodoViewModel", e.message.toString())
            _todos.value = emptyList()
        } finally {
            _loading.value = false
        }
    }
}
```

3. Por último, modifique el componente **TodoScreen** agregando los componentes necesarios para capturar la información de la nueva tarea.

```
package com.example.todocompose.ui.theme.composables

import androidx.compose.foundation.layout.*
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.dp
import com.example.todocompose.TodoViewModel

@Composable
fun TodoScreen(viewModel: TodoViewModel) {
    val todos by viewModel.todos.collectAsState()
    val loading by viewModel.loading.collectAsState()

    var titleInput by remember { mutableStateOf("") }
    var descriptionInput by remember { mutableStateOf("") }

    LaunchedEffect(Unit) {
        viewModel.loadTodos()
    }

    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(16.dp),
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Text(
            text = "Mis Tareas",
            style = MaterialTheme.typography.headlineMedium,
            modifier = Modifier.padding(bottom = 16.dp, top = 16.dp)
        )

        TextField(
            value = titleInput,
            onValueChange = { titleInput = it },
            label = { Text("Titulo") },
            modifier = Modifier.fillMaxWidth()
        )

        Spacer(modifier = Modifier.height(8.dp))

        TextField(
            value = descriptionInput,
            onValueChange = { descriptionInput = it },
            label = { Text("Descripcion") },
            modifier = Modifier.fillMaxWidth()
```

```
        )

        Spacer(modifier = Modifier.height(8.dp))

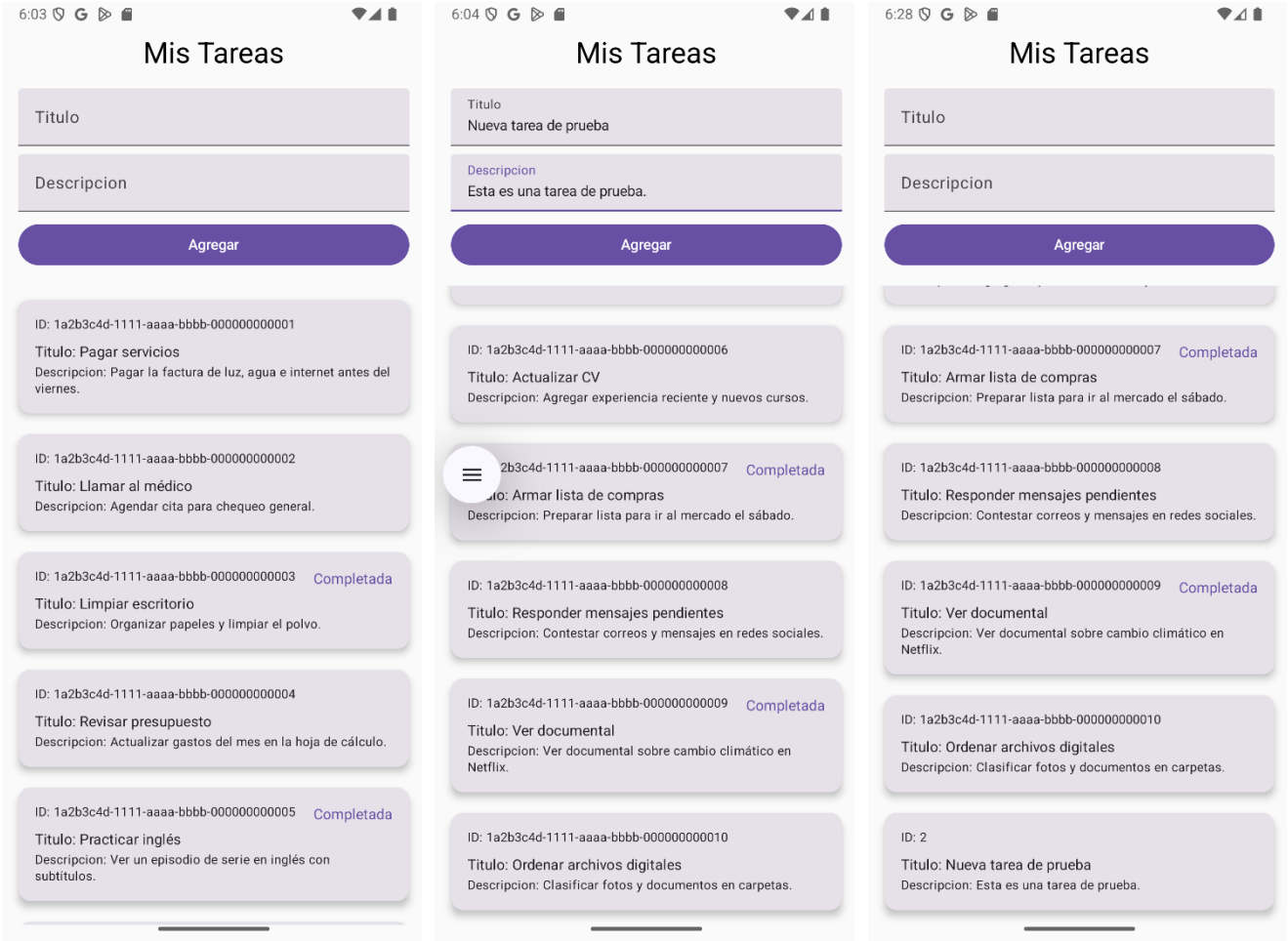
        Button(
            onClick = {
                viewModel.addTodo(titleInput, descriptionInput)
                titleInput = ""
                descriptionInput = ""
            },
            modifier = Modifier
                .fillMaxWidth()
        )

        {
            Text("Agregar")
        }

        Spacer(modifier = Modifier.height(16.dp))

        if (loading) {
            CircularProgressIndicator()
        } else {
            TodoList(todos = todos)
        }
    }
}
```

4. Guarde los cambios y ejecute nuevamente la aplicación.



IV. DISCUSIÓN DE RESULTADOS

Complemente el funcionamiento de la aplicación agregando las opciones:

- Marcar como tarea completada.
- Modificar tarea.
- Eliminar la tarea.

NOTA: **Asegúrese de solo modificar o eliminar los registros creados por usted.**

V. BIBLOGRAFÍA

- Como comenzar con Jetpack Compose
  https://developer.android.com/develop/ui/compose/documentation?hl=es-419

- Instructivo de Jetpack Compose
  https://developer.android.com/develop/ui/compose/tutorial?hl=es-419

- Ejemplos de Compose
  https://developer.android.com/develop/ui/compose/samples?hl=es-419